Glass Heroes v0.3.4 Issue #37 Placement & Parking Experimental

October 24, 2026. EXPERIMENTAL — FINAL OWNER ACCEPTANCE PENDING YES/NO. GTA testing NOT RUN. Windows execution NOT RUN. Stack-overflow cause UNATTRIBUTED; this is a feature candidate, not a crash fix. RC2 diagnostic testing remains independent.

Source and authorization

Owner pre-approved A–D implementation in chat after the initial proposal-only request. That authorization superseded the initial stop before implementation. Issue #37's second owner YES/NO acceptance gate remains pending after this complete review package. No live GTA modification, main merge or public release is performed.

Branch: feature/shvdn3-37-placement-parking. Parent: 75e3623f608a05d1a2b59d12c243dbcdfc061358. It contains the verified RC1 source at e312d8c9500f4a990eb993b82e8e76b55711d3de, SHA256 70bc95bcf0d16cbf36b61870bff7e21193c05479b79a4d8beeb45a0b734c2120.

Final A–D code checkpoint: f6b90560304033ca538b585e61f850ee50863e10. Candidate source SHA256: d9e7eb3ed5327acc6d61d368510a50d6d47df52dfab2caba2acd74e438749236. FINAL_HANDOFF.md/JSON in the ZIP records the exact final branch tip including the documentation commit; this guide identifies the code checkpoint to avoid a circular commit hash.

Individual focused commits:

- A: 5f82f5b7979b95800c3bba4b6174705c4dba260f — direction-free customer handoff.

- B: 8d794ddded8a7229565ea91ba71b5ac0e495de37 — walking-point terminology and focus.

- C: cb2704601203991981fef612610f5bef70ba4ee0 — persistent marker filters and cached previews.

- D: f6b90560304033ca538b585e61f850ee50863e10 — protected reset/restore and complete regression evidence.

Original RC1, maintenance, main and RC2 diagnostic source/branches are untouched. No RC2 CustomerVehicleTrace instrumentation is present in this candidate. RC1's existing generic diagnostic logger remains unchanged. Source retains the existing DEV/EXPERIMENTAL header; identify the candidate by this branch, package and SHA256. This is not v0.3.5.

Menu hierarchy and labels

Placement Settings retains Customer Locations, Employee Locations, Service Bays, Delivery & Stockroom and Walking Routes & Approaches. It adds Reset & Restore Placements beside Marker Display Settings.

Walking Routes > Mechanics identifies:

- Staff Spawn -> Break: five shared physical entry-path stops; the existing car pickup/return reuse toggle controls reversed pickup and forward return traversal.

- Break <-> Service Bay Stops: five shared physical pedestrian points.

- Final Walking Approaches: per-workstation pedestrian approaches, separate from car parking positions.

- Stops Used by Each Mechanic > mechanic > To Service Bay / From Service Bay: existing per-direction masks choose which shared physical stops to use.

- Advanced: Legacy Checkpoint and the existing walking options.

Route-stop values show USE or SKIP. The description independently says the physical coordinate is SAVED or NOT SAVED. Preview/count/header wording follows this distinction. Selecting a row does not edit coordinates or masks. Existing outbound/return order, modes, fallback routing and EffectiveWorkRouteMask remain unchanged. Focused marker display resolves the exact physical point in the list, mechanic selection or PointEditor; unset points do not receive invented markers.

Marker Display has Display Mode, Hide All, Show Current Category, Restore Last Custom, five category switches, separate computed auto-bay preview, selected-point identity and Back. Modes are Hide All, Focused, Custom and All. Categories are Customers & Queue, Mechanics & Walking, Service Vehicle Bays, Receptionist/Manager/Equipment and Supplier & Stockroom. Hide All gates even computed previews and selected highlights. A saved point never forces display ON. PointEditor links to Marker Display; choosing Focused provides the selected-point preview without changing AI settings.

Reset & Restore has six category resets (Customers, Mechanics, Service Vehicle Bays, Receptionist/Manager, Supplier/Stockroom, Equipment Positions), optional shop-anchor inclusion for Full Reset, Reset All and Restore from Backup. Restore offers a scope selector, optional anchor for All scope, latest-backup status and review. Both confirmation pages default NO. Back, pause, F2, expiry and context changes cancel the pending request. The existing keyboard/controller arbiter remains in control.

Actual production menu inventory and descriptions emitted by the final navigation harness are in NAVIGATION_MATRIX.md and MENU_FOOTERS.md. They are source-derived evidence, not screenshots or native rendering qualification.

A — actual direction-free parking behavior

Three heading-forcing drop-off task paths are addressed: customer arrival parking in UpdateOneCustomerOrder, mechanic return parking in UpdateVehicleApproach and the drop-off retry in UpdateVehicleParking. New BeginDirectionFreeHandoff/IssueDirectionFreeHandoff/UpdateDirectionFreeHandoff share a position-only slow TASK_VEHICLE_DRIVE_TO_COORD_LONGRANGE task at 2.4m/s, careful driving flags and 1.75m stop range. No saved-facing argument is supplied. The original fine-parking option still chooses the 14m versus 7m final-approach trigger for intake.

Handoff requires the existing 4m center acceptance zone, finite nonnegative vehicle speed below 1.5m/s, two consecutive low-speed observations on the existing approximately one-second state cadence, and existing vehicle-occupancy clearance excluding the order's own car. An occupied zone delays task/exit; interruption resets the stable-stop samples. At most one position-only no-progress retry after 14 stable seconds. Customer final approach uses the existing parking timeout; mechanic drop-off uses parking timeout +15 seconds. The original road-arrival/outer drive limits remain. The old timeout fallback that could exit a moving car within 7m is removed. There is no forced acceleration, instant braking, occupied-car teleport or new world scan per frame.

Unpaid failures use existing cancellation/promise retention; paid return failures use existing safe paid-return recovery. Settlement, reservations, invoices, stock and handoff ownership are unchanged. Service-bay TASK_VEHICLE_PARK facing, retry behavior and queued-car directional parking are unchanged. Existing empty-car pickup transfer safety/idempotence remains; pickup no longer rotates to drop-off heading. Service-bay teleport still applies bay heading. Drop-off heading stays in the save for compatibility and initial non-driving spawn presentation; it is not a handoff requirement.

The drop-off marker is an orientation-neutral 8m-diameter floor zone. Service/queue parking retains directional outlines. Removing heading from a task cannot guarantee GTA road/navmesh success in a particular MLO; that requires owner testing.

B/C — cosmetic state and recurring performance

PlacementDisplay.v1.ini is a new, separate cosmetic preference file under the existing business-data folder. Version=1, Mode=0..3, ActiveMask=0..31 and CustomMask=0..31 are its only new keys. Defaults: Hide All and all five category bits retained for Custom. Existing Save.ini/Settings.ini keys/defaults/schema are unchanged. No template preference file is supplied to overwrite a user's data. Writes occur only on explicit mode/filter changes using the existing atomic file-save helper. Failed writes retain the prior visible state; unsupported/oversized preferences fail closed and preserve the file.

Show Current Category can temporarily select one active category without overwriting remembered Custom. Restore Last Custom restores that mask. Computed auto-bay preview remains a session-only explicit toggle, respects Hide All/category filters and does not change parking mode or geometry. Focused limits its preview to the selected bay. Saving/clearing/editing a point does not override the user's marker mode.

The 58-entry registry holds existing point references, labels, category/group and marker shape. There are no spawned marker entities or per-frame model requests/world searches. Drawing keeps the 80m shop/point range and pause guards, skips nonfinite points/headings, caches parking outlines and recomputes automatic-bay previews only when their inputs change. Colors and registry are cached. No recurring preference reads/writes. Displaying a saved but inactive route never enables that route.

D — reset/backup/restore safety

Both review and final confirmation must be completed within the pending 30-second review lifetime. Both default NO; first YES enters a second page at NO. Commit also requires the reviewed-stage flag. No data mutation merely from browsing/reviewing. The current save hash must still match the review. Safety is checked again before commit.

Eligibility requires startup complete, active unpaused game, shop CLOSED, no remote mode, no active/staged/current/paid-return customer, no queued request, no saved repair promise, no remote job and no mechanic background job. Any pending supplier order, carried/loaded/unloaded cargo or still-owned supplier vehicle/driver/crate blocks the operation. The action never cancels an obligation to become eligible. A post-commit cleanup problem blocks reopening until reload.

Only explicitly selected spatial keys are overlaid: Set/X/Y/Z/Heading for category point sections, affected mechanic/receptionist resume posts, optional Shop Placed/X/Y/Z and equipment placement flags. Route masks/modes, money, stock, invoices, receipts, personnel, wages, ownership/upgrades, pallet purchases, delivered-package masks, display preferences and fixed SupplierWarehousePickup remain. Equipment positions may clear; purchased benefits remain. Purchased pallets are not repurchased after reset; their cleared positions must be placed again. A pallet reset that would leave stock without capacity is blocked. An unrelated category can reset without altering an already over-capacity legacy inventory. Restore cannot leave its selected pallet layout short of current stock.

Restored set points must lie within 80m of the effective saved/current shop anchor. A mismatched layout requires the matching All-scope anchor or another layout; no actor is moved to invalid/distant coordinates to force acceptance. Moving a configured shop more than 20m no longer silently clears its placements; the existing location action directs that operation through protected Reset & Restore.

Commit sequence:

1. Revalidate both-stage consent, lifecycle and review save hash. Flush a normal consistent business snapshot using unchanged Save logic; no geometry changed yet.

2. Copy the COMPLETE business-data directory and matching installed source to Documents/GlassHeroes/PlacementBackups outside script-loading paths. Copy source as inert Source.cs.txt; include SHA256/length manifest and verify every copy and original before/after copying. Never omit a file silently. Reparse/linked paths, missing source/save, unsupported size, changing files or failed writes abort. Automatic limits: 512 enumerated files, 256 directories, 64MiB data enumeration and 4MiB source/save limits; aggregate verification can impose a lower effective data allowance once manifest/source are included. Older complete backups are not deleted. More than 256 backup folders requires manual archiving before browsing.

3. For restore, verify the selected backup and unchanged manifest again. Build typed spatial changes, validate finite values, scope, anchor and capacity. Restore overlays geometry onto CURRENT business data; it never restores old cash/stock/ledger from the snapshot.

4. Patch only whitelisted spatial values. Preserve unrelated keys/comments/text, UTF-8/BOM/UTF-16 encoding and line endings. Duplicate/ambiguous INI keys fail closed. Stage/flush/read-verify a same-directory temporary file, check the current save hash immediately before atomic File.Replace, then commit. Do not edit the save externally during this explicit operation; the last-moment hash check is not an OS transaction with arbitrary independent editors.

5. Only after durable commit apply the typed coordinates/flags in memory, invalidate cosmetic caches and clean only affected owned visuals. Pre-commit failure keeps geometry. If cleanup is interrupted after commit, the new save remains authoritative; reopening is held pending reload. No unsafe automatic whole-ledger rollback follows a successful atomic replace. The complete pre-operation backup remains available for manual closed-game recovery.

The backup is synchronous, bounded and runs only after final confirmation while the shop is closed; copying a large directory can briefly stall menu responsiveness. No background thread, script yield/reentry, periodic backup or per-frame disk work is added. Checksums detect corruption; backups are not authenticated imports from arbitrary third parties. Use only the owner's verified backups and inspect scope/path before consent.

Source-level scope and test evidence

Only experimental/shvdn3/Install/scripts/GlassHeroesBusiness.3.cs changes gameplay source. Matching Visual Studio project/solution and original v2 sources remain byte-identical to RC1. Source diff and excerpts are included; applying source.diff to the pinned RC1 reproduces this candidate exactly. final-validation/preservation.json lists all 39 changed existing members and 49 added members; 397 original members and all original declarations/defaults outside approved blocks are unchanged.

The audit freezes Save, finance/quote/receipt/inventory/supplier methods, original Tick/time sampling, BuildRoute and saved mask mutation. It checks unchanged service-bay/queue behavior, the allowed pickup-heading-only delta, individual point mutation behavior apart from visibility, and input logic apart from cancelling a new review. RC2 trace types/prefixes are absent. Likely RC2 conflicts are the three transfer methods both variants touch; no combined candidate is produced. A future integration requires separate authorization, deliberate reconciliation and a new complete build/regression/GTA qualification.

Actual final clean Release: PASS, 0 errors / inherited warnings CS0169 and CS0649. SDK 8.0.426; net48/C#7.3; official ScriptHookVDotNet3 package 3.6.0. Behavior/navigation 1057/1057 PASS: inherited RC1 954 retained, 88 new cases (parking37 18, focus37 9, markers37 24, reset37 37), and navigation expanded from 212 to 227. Route tests retain 6144 mask/saved-mask/direction/worker combinations inside their existing 16 cases; do not count those as 6144 extra tests. Static21/21, inventory structure13/13 and preservation PASS.

Inherited fixtures retain their original accounting/input assertions. Adapters supply new session types, cancellation hooks or menu-caption context. Only deliberately changed expectations are pickup heading preservation, direction-free return retry, approved labels and expanded menu rows/modes. New tests exercise the actual new helpers separately using deterministic native adapters and real

synthetic temporary file operations. A/B/C targeted evidence and the complete final command logs are included. A superseded attempt stopped on a missing payroll-fixture caption context; its folder is explicitly labelled superseded, and the complete corrected run passed.

The full final suite ran before the D code commit; validation.source_revision identifies the then-current C checkpoint, while source_sha256 identifies the actual D bytes compiled/tested. Report/package generation checks equality to the final code commit and hash. Documentation-only commits do not change product qualification. The ZIP receipt records independent extraction/hash verification and another clean actual product compilation.

Reproduce from the full Git checkout (Python 3/.NET 8 tooling; cloud shell: source /workspace/.cloud-setup/env.sh):

```
python verification/inventory340/validate.py --output /tmp/issue37-review
--issue36-integration --integration-approved --issue37
```

The integration flag refers to already accepted Issue #36 in RC1, not acceptance of Issue #37. Full preservation needs Git history/protected baseline files; archive test sources/evidence are references. Source/project rebuild from the ZIP:

```
dotnet build VisualStudio/GlassHeroesBusiness.V3.Experimental.csproj -c Release
```

Automated adapters do not prove GTA task outcomes, MLO clearance, real controller behavior, rendering/performance or crash stability. GTA testing NOT RUN; Enhanced/runtime compatibility NOT VERIFIED. Proceed only with the separate testing/rollback instructions, protecting active RC2 diagnostics. Final owner acceptance remains pending YES/NO.

Issue37 isolated testing and rollback reference

EXPERIMENTAL. GTA testing NOT RUN. Second owner acceptance gate PENDING YES/NO. No installation has been performed by Codex. Do not overwrite the active RC2 diagnostic candidate or stable RC1. No feature is a verified crash correction.

1. Preserve the active RC2 diagnostic installation, exact source, complete matching business data and all crash logs. Use a separate GTA test copy/profile with a disposable complete business-data copy for Issue37, or explicitly coordinate an owner-controlled testing window after the RC2 session is closed. Never run both variants/generations at once.

2. With GTA/script hosts closed, back up the exact prior script and COMPLETE scripts/GlassHeroesBusiness directory together outside every script-loading path. Include Save.ini, Settings.ini, CSVs, profiles, logs and all companion files. Do not alternate candidate sources against the sole modified live save.

3. In the isolated test profile only, install Install/scripts/GlassHeroesBusiness.3.cs as scripts/GlassHeroesBusiness.3.cs. Do not copy the whole ZIP or its test/documentation folders into GTA. No runtime DLL, Save.ini, Settings.ini or default preference file is supplied. Exactly one Business Manager source/generation/DLL must be active.

4. Verify the source SHA256: d9e7eb3ed5327acc6d61d368510a50d6d47df52dfab2caba2acd74e438749236. Use Get-FileHash on Windows and compare SOURCE_SHA256.txt/PACKAGE_MANIFEST.json. Source retains its existing DEV label; use the hash/branch to identify Issue37. Compile API is SHVDN3 3.6.0/net48/C#7.3; the owner's GTA/Enhanced/mod-stack compatibility remains unverified.

5. Run GTA_TEST_CHECKLIST.md and preserve complete pre/post-test data and logs. Test resets/restores ONLY on disposable data. A confirmed runtime operation writes a full verified backup to Documents/GlassHeroes/PlacementBackups; it refuses missing source, linked paths, changing files, over-limit data, outstanding obligations or insufficient stock capacity. Do not bypass these guards or delete business data to make a reset eligible.

6. New cosmetic PlacementDisplay.v1.ini is created only on an explicit display change. Default is Hide All. Point saves preserve the mode. No business-schema migration is introduced.

For in-game undo, review the latest backup path/status and selected scope under Reset & Restore. Two confirmations are required; the operation first backs up CURRENT data and then restores only chosen geometry. It preserves current money/stock/ownership/masks. Use All with optional matching shop anchor only when appropriate. Geometry outside 80m of the effective shop anchor or a pallet layout too small for current stock is rejected. Verify layout after restore before reopening.

For full rollback, close GTA/script hosts. Archive candidate source, complete candidate data and logs. Restore the exact previous script PLUS its COMPLETE matching pre-test data/settings backup, including companion journals/CSVs/profiles. Do not mix old Save.ini with newer ledger/receipt files or reset the sole save. Ensure exactly one Business Manager source is active and verify its known hash. Keep RC2 evidence untouched; resuming RC2 uses its exact source/hash and matching data set.

If interrupted after a placement commit but during visual cleanup, do not attempt an automatic ledger rollback. The committed save is authoritative and reopening is held pending reload. Preserve the backup/evidence, reload with the same candidate in the disposable profile and inspect, or perform the closed-game full rollback above. Nothing in this archive deploys, restores or deletes live files automatically.

Issue37 GTA acceptance checklist — ALL NOT RUN

Protect RC2 testing. Use an isolated profile/copy, disposable complete business data, exact source/package hashes and unchanged heavy-mod configuration. Record every result, local time/timezone, game/SHVDN versions, input device and screenshots/logs. The recurring stack-overflow cause remains UNATTRIBUTED.

1. Cold launch/reload: unchanged startup delay, one script, correct source hash, preserved balances/stock/staff/invoices/receipts/promises/settings and coordinates/masks. Marker mode defaults Hidden or loads the existing separate preference file; no point is reset on load.

2. Incoming customer drive from north/east/south/west and opposite curb/driveway orientations. Test fine approach ON/OFF. Confirm the car approaches/stops safely without aligning to saved drop-off facing, exits only when slow and within the zone, then checks in normally. Observe MLO floors, road obstacles, traffic and player proximity.

3. Mechanic return of a repaired car from both orientations in CUSTOM and AUTO LOT. Confirm direction-free drop-off, existing driver exit/pedestrian return and customer pickup/departure. Exactly one invoice/stock consumption/payment; no lost paid return or car.

4. Service bays and queued-car spaces still use saved orientation, existing clearance, bounds and retries. Test an occupied drop-off, stopped obstruction, slow/no progress, timeout, missing driver and paid recovery in controlled disposable scenarios. No heading-forcing task, task spam, forced rolling exit or occupied-car warp should appear. Native adapter success does not guarantee real pathfinding.

5. Mechanic menus distinguish Staff Spawn -> Break, Break <-> Service Bay physical stops, final pedestrian approaches and per-mechanic To/From Bay USE/SKIP. Check saved versus missing coordinates, sparse/different masks, DIRECT inactive status and long worker names. Select each physical stop and confirm Focused highlights that exact point; browsing never saves/moves anything.

6. Marker modes: Hide All, Focused, Custom, All; test every category, Show Current Category, Restore Last Custom, selected-point identity and separate auto-bay preview. Hide All must hide every preview. Save/update/clear points with Hidden/Custom and confirm the preference is not overridden. Reload and verify preference persistence. Check floor height, opposite/cardinal outline arrows for service/queue bays, neutral drop-off zone, range and clutter.

7. Keyboard-only, controller-only and mixed input: all new pages, paging beyond ten rows, Back restores caller/row, held Enter cannot cross both confirmation pages, frontend echoes do not double-act, F2/Escape/native pause cancel pending reset consent. Confirm current menu appearance/footer readability remains intact at 720p/1080p and the owner's resolution/overlays.

8. Reset negative paths using copied data: boot/pause/open shop, live/staged/queued customer, unpaid promise, paid-return, background mechanic job, remote job/mode, supplier order/carry/cargo/leaving owned actors. Every action must refuse without cancelling an obligation. Supplier/full reset with stock must refuse if capacity would be lost. Review/NO/Back/expiry must not create a reset or backup.

9. Safe closed/idle reset: inspect count/scope, leave anchor excluded, pass two deliberate YES steps, verify complete data/source manifest backup outside loading paths. Reset each category separately and verify ONLY its geometry/resume posts/appropriate equipment placement flags change. All finances, stock, invoices, personnel, purchases, masks/modes, delivered masks, unknown settings and fixed world warehouse pickup remain.

10. On disposable data, full reset excluding and including anchor. Confirm no orphaned owned visuals or invalid references; reconfigure cleared positions before opening. Purchased pallets/equipment remain owned, without repurchase; empty pallet positions need placement. Moving an already configured shop more than 20m must refuse silent clearing and point to protected reset.

11. Restore latest verified placement backup with selected categories and full scope. Change copied financial data through normal gameplay between reset and restore; confirm restore changes geometry without reverting those legitimate finances/stock/receipts/masks. Verify enough owned pallet capacity, matching anchor/range and complete new pre-restore backup. Never hand-edit the sole live save to force a test.

12. Where safely testable in the isolated profile, missing/denied/over-limit/corrupt backup or stale save must abort before geometry changes. Do not modify backups/live permissions to force failures in the active installation. A post-commit cleanup failure must hold reopening pending reload and retain the durable new geometry/backup.

13. Pause/resume, menu dwell, distant streaming, save/reload and game close/reopen: active-unpaused progression only, no offline income/ETA/wage catch-up; unchanged physical wholesale player collection and supplier receipts; no duplicate entities/accounting.

14. Compare performance during Hidden, Focused, Custom and All. Stable marker/auto-bay inputs should not cause repeated geometry/model work; no recurring file I/O. Explicit complete backup may briefly stall input. Preserve any crash/host/dump evidence with exact source/mod/runtime/time context; do not infer crash attribution from the last visible car operation.

After owner testing/review, request the second explicit YES/NO acceptance. No merge into RC2, RC1 or main follows automatically.

v0.3.4 Issue #37 Placement & Parking Experimental

October 24, 2026. Implementation pre-approved; final acceptance PENDING owner YES/NO. GTA testing NOT RUN.

- A: position-only slow customer drop-off approach for incoming/returned vehicles, stable low-speed/clearance acceptance and bounded retry/timeout. Preserve service/queue bay orientation and paid-return accounting. Neutral drop-off zone; saved heading retained for compatibility/initial spawn.

- B: plain Staff Spawn -> Break and Break <-> Service Bay labels, final pedestrian approaches, per-mechanic use separate from saved coordinates, exact selected-point focus. No saved mask/route geometry change.

- C: persistent cosmetic Hide All/Focused/Custom/All with five categories, custom recall and filtered cached auto previews. Point saves no longer force visibility; separate UI preference file.

- D: guarded per-category/full geometry reset and scope-based restore, opt-in full shop anchor, two default-NO confirmations, complete verified backup, atomic spatial patch, current-ledger preservation and cleanup/reload guard. Remove implicit destructive clearing when relocating a configured shop.

- Actual Release: 0 errors/two inherited warnings. 1057/1057 behavior/navigation, 21/21 static, 13/13 structural and preservation PASS. Word/PDF, source diff, manual GTA/rollback and exact final handoff included.

- No main/RC1/RC2 modification, merge, public release, live installation or crash-fix claim. Not v0.3.5.

Issue37 before/after excerpts

Complete exact patch: source.diff. Base and code checkpoint are identified in the developer guide.

UpdateVehicleParking

Before

```
    private void UpdateVehicleParking(CustomerOrder order, bool atBay)
    {
        WorkBay dest = VehicleDestination(order, atBay);
        if (DistanceSquared(order.Vehicle.Position, dest.Position) < 16f &&
order.Vehicle.Speed < 1.5f)
        {
            Ped worker = AssignedWorker(order);
            if (worker == null) { FailVehicleTransfer(order, atBay, "Mechanic missing at
parking"); return; }
            GTA.Native.Function.Call(GTA.Native.Hash.TASK_LEAVE_VEHICLE,
                worker.Handle, order.Vehicle.Handle, 0);
            order.Stage = atBay ? CustomerStage.MechanicExitingAtBay :
CustomerStage.MechanicExitingAtDropoff;
            order.SecondsInStage = 0;
        }
        else if (order.SecondsInStage > _parkingTimeoutSeconds + 15)
        {
            Ped worker = AssignedWorker(order);
            if (order.ParkingRetries == 0 && worker != null && MechanicInCustomerCar(order))
            {
                order.ParkingRetries++; order.SecondsInStage = 0;
                GTA.Native.Function.Call(GTA.Native.Hash.TASK_VEHICLE_PARK, worker.Handle,
order.Vehicle.Handle,
                    dest.Position.X, dest.Position.Y, dest.Position.Z, dest.Heading, 1, 2.8f,
false);
                Log("Parking retry #" + order.Id + " direction=" + (atBay ? "BAY" : "PICKUP")
+ " paid=" + order.Paid, null);
            }
            else FailVehicleTransfer(order, atBay, "Parking failed at saved point; review
heading/clearance");
        }
    }
```

After

```
    private void UpdateVehicleParking(CustomerOrder order, bool atBay)
    {
        if (!atBay) { UpdateDirectionFreeHandoff(order, true); return; }
        WorkBay dest = VehicleDestination(order, atBay);
        if (DistanceSquared(order.Vehicle.Position, dest.Position) < 16f &&
order.Vehicle.Speed < 1.5f)
        {
            Ped worker = AssignedWorker(order);
            if (worker == null) { FailVehicleTransfer(order, atBay, "Mechanic missing at
parking"); return; }
            GTA.Native.Function.Call(GTA.Native.Hash.TASK_LEAVE_VEHICLE,
                worker.Handle, order.Vehicle.Handle, 0);
            order.Stage = atBay ? CustomerStage.MechanicExitingAtBay :
CustomerStage.MechanicExitingAtDropoff;
            order.SecondsInStage = 0;
        }
        else if (order.SecondsInStage > _parkingTimeoutSeconds + 15)
        {
            Ped worker = AssignedWorker(order);
            if (order.ParkingRetries == 0 && worker != null && MechanicInCustomerCar(order))
            {
                order.ParkingRetries++; order.SecondsInStage = 0;
```

```
                GTA.Native.Function.Call(GTA.Native.Hash.TASK_VEHICLE_PARK, worker.Handle,
order.Vehicle.Handle,
                    dest.Position.X, dest.Position.Y, dest.Position.Z, dest.Heading, 1, 2.8f,
false);
                Log("Parking retry #" + order.Id + " direction=" + (atBay ? "BAY" : "PICKUP")
+ " paid=" + order.Paid, null);
            }
            else FailVehicleTransfer(order, atBay, "Parking failed at saved point; review
heading/clearance");
        }
    }
```

ConfigureRoutePoint

Before

```
    private void ConfigureRoutePoint(WorkBay point, string label)
    {
        if (!_placed) { Notify("Set your shop location first."); return; }
        if (_clockedIn) { Notify("Close the shop before editing employee paths."); return; }
        if (Game.Player.Character == null) return;
        Vector3 here = Game.Player.Character.Position;
        if (DistanceSquared(here, _location) > 6400f)
        { Notify("Route markers must be within 80m of the shop."); return; }
        if (point.IsSet && DistanceSquared(here, point.Position) < 2.25f)
        {
            point.IsSet = false;
            Save();
            Notify(label + " CLEARED.");
            return;
        }
        point.IsSet = true;
        point.Position = here;
        point.Heading = Game.Player.Character.Heading;
        Save();
        _showPlacementMarkers = true;
        Notify(label + " SAVED: " + FormatPosition(here) + ". Markers ON.");
    }
```

After

```
    private void ConfigureRoutePoint(WorkBay point, string label)
    {
        if (!_placed) { Notify("Set your shop location first."); return; }
        if (_clockedIn) { Notify("Close the shop before editing employee paths."); return; }
        if (Game.Player.Character == null) return;
        Vector3 here = Game.Player.Character.Position;
        if (DistanceSquared(here, _location) > 6400f)
        { Notify("Route markers must be within 80m of the shop."); return; }
        if (point.IsSet && DistanceSquared(here, point.Position) < 2.25f)
        {
            point.IsSet = false;
            Save();
            Notify(label + " CLEARED.");
            return;
        }
        point.IsSet = true;
        point.Position = here;
        point.Heading = Game.Player.Character.Heading;
        Save();
        Notify(label + " SAVED: " + FormatPosition(here) + ".");
    }
```

SetShopLocationFromPlayer

Before

```
    private void SetShopLocationFromPlayer()
    {
        if (_clockedIn) { Notify("Close the shop before moving its location."); return; }
        if (Game.Player.Character == null) return;
        Vector3 newLocation = Game.Player.Character.Position;
```

```
        if (_placed && DistanceSquared(_location, newLocation) > 400f)
        {
            foreach (WorkBay bay in _bays) bay.IsSet = false;
            _employeeEntrance.IsSet = false;
            _walkCheckpoint.IsSet = false;
            foreach (WorkBay point in _arrivalRoute) point.IsSet = false;
            foreach (WorkBay point in _receptionRoute) point.IsSet = false;
            _breakArea.IsSet = false;
            _receptionDesk.IsSet = false;
            _receptionApproach.IsSet = false;
            _customerDriveSpawn.IsSet = false;
            _customerDropoff.IsSet = false;
            _customerApproach.IsSet = false;
            _customerQueue.IsSet = false;
            _queueCar2.IsSet = false;
            _queueCar3.IsSet = false;
            foreach (WorkBay point in _serviceRoute) point.IsSet = false;
            foreach (WorkBay point in _bayApproaches) point.IsSet = false;
            Notify("New shop: bays, entrance, break and walking routes cleared.");
        }
        _location = newLocation;
        _placed = true;
        Save();
        Notify("Shop location SET: " + FormatLocation() + ". " +
            (File.Exists(SaveFile) ? "Save.ini exists." : "WARNING: Save.ini missing!"));
    }
```

After

```
    private void SetShopLocationFromPlayer()
    {
        string blocked = PlacementOperationBlock();
        if (blocked != null) { Notify(blocked); return; }
        if (Game.Player.Character == null) return;
        Vector3 newLocation = Game.Player.Character.Position;
        if (_placed && DistanceSquared(_location, newLocation) > 400f &&
PlacementScope(6).Exists(point => point.Target.IsSet))
        { Notify("Moving this configured shop would invalidate placements. Use Reset & Restore
 with the optional shop anchor first; nothing cleared."); return; }
        Vector3 previous = _location; bool placed = _placed;
        _location = newLocation; _placed = true;
        if (!Save()) { _location = previous; _placed = placed; return; }
        Notify("Shop location SET: " + FormatLocation() + ". Existing placements preserved.");
    }
```

New helper: IssueDirectionFreeHandoff

Before: no corresponding helper. After:

```
    private void IssueDirectionFreeHandoff(CustomerOrder order, Ped driver)
    {
        Vector3 p = _customerDropoff.Position;
        // Position-only, slow, traffic-aware task. No saved-facing parameter.
        GTA.Native.Function.Call(GTA.Native.Hash.TASK_VEHICLE_DRIVE_TO_COORD_LONGRANGE,
            driver.Handle, order.Vehicle.Handle, p.X, p.Y, p.Z, 2.4f, CarefulDrivingStyle,
1.75f);
        order.Handoff.Issued = true;
        order.ParkingTaskIssued = true;
        order.Handoff.LastPosition = order.Vehicle.Position;
        order.Handoff.NoProgress = 0;
    }
```

New helper: PlacementCategoryVisible

Before: no corresponding helper. After:

```
    private static bool PlacementCategoryVisible(PlacementDisplayMode mode, int mask, int
category, bool focused)
    {
        if (mode == PlacementDisplayMode.Hidden) return false;
        if (mode == PlacementDisplayMode.Focused) return focused;
        return mode == PlacementDisplayMode.All || (category >= 0 && category < 5 && (mask &
```

```
(1 << category)) != 0);
    }
```

New helper: TryCommitPlacementBytes

Before: no corresponding helper. After:

```
    private static bool TryCommitPlacementBytes(string path, string expectedHash, byte[]
candidate, Action<string, string> replace, Action<Exception> report)
    {
        string temporary = null;
        try
        {
            path = Path.GetFullPath(path); RejectPlacementReparsePath(path);
            temporary = Path.Combine(Path.GetDirectoryName(path), ".gh-placement-" +
Guid.NewGuid().ToString("N") + ".ini");
            using (var output = new FileStream(temporary, FileMode.CreateNew,
FileAccess.Write, FileShare.None))
            { output.Write(candidate, 0, candidate.Length); output.Flush(true); }
            if (!File.ReadAllBytes(temporary).SequenceEqual(candidate) || PlacementHash(path)
!= expectedHash) throw new IOException("Save changed while preparing placements; operation
aborted.");
            replace(temporary, path); temporary = null; return true;
        }
        catch (Exception error) { try { report(error); } catch { } return false; }
        finally { if (temporary != null) try { File.Delete(temporary); } catch (Exception
error) { try { report(error); } catch { } } }
    }
```